University of Mindanao

Department of Computing Education

CST4/L: CS Professional Track 1

Automata and Language Theory

Submitted By:

Name	Student ID	Signature	Date
Bai Fatima Andong	545438		
Fionnah Keiz Coyoca	545499		
Christian James Cahilig	544797		
Karylle Mish Gellica	544337		
John Llorie Sarmiento	545495		

Instructor:

Modesto C. Tarrazona

Submission Date:

17 – 12 – 2024

Table of Contents

1. Introduction

This project addresses the challenge of understanding and visualizing fundamental concepts in automata theory and computation. Automata theory is a core topic in computer science, yet its abstract nature often makes it difficult for learners to grasp. To bridge this gap, the program offers users the ability to engage with predefined models, including Deterministic Finite Automata (DFA), Non-Deterministic Finite Automata (NDFA), Pushdown Automata (PDA), Context-Free Grammars (CFG), the Tower of Hanoi problem, and Turing Machines. Each model is equipped with predefined languages and scenarios to help users visualize and understand the behavior of these computational systems.

Its scope extends to anyone interested in exploring automata theory. This comprehensive tool bridges the gap between theory and application, offering valuable insights into the behavior of these fundamental systems.

2. Objectives

The objective is to provide a straightforward program that allows users:

- To experiment with key computational models (DFA, NFA, CFG, PDA, Turing Machine, Tower of Hanoi).
- To test and validate strings, production rules, and operations to understand the models' behavior.
- To visualize practical examples such as string validation, binary operations, and formal transitions, bridging the gap between abstract theory and real-world applications.

3. Background and Literature Review

Automata theory is a foundational area of computer science that focuses on abstract machines (automata) and the computational problems they can solve. It provides the theoretical framework for understanding formal languages, which are sets of strings defined by specific rules or patterns (Sipser, 1996). Key models in automata theory include:

- **Deterministic Finite Automata (DFA)**
  A state machine used to determine whether a given string belongs to a specific regular language. DFA operates with a single path of computation for each input string (Booth, 1967).

- **Non-Deterministic Finite Automata (NFA)**
  Similar to DFA but allows multiple paths of computation, making it easier to design for certain languages. Both DFA and NFA recognize regular languages (Hopcroft & Ullman, 1979).

- **Context-Free Grammars (CFG)**
  A formal grammar used to generate strings in context-free languages. CFGs are commonly used in parsing and language design (Chomsky, 1956).

- **Pushdown Automata (PDA)**
  A computational model with a stack, capable of recognizing context-free languages, such as those requiring nested structures (Hopcroft & Ullman, 1979).

- **Turing Machines**
  A more powerful abstract machine that can simulate any computation, used to represent general algorithms and the limits of computation (Turing, 1936; Gandy & Yates, 2001).

- **Tower of Hanoi**
  A classic algorithmic problem that demonstrates recursive problem-solving and computational efficiency.

Previous Works

Automata theory has been studied extensively since its introduction by Alan Turing and others in the 20th century. Deterministic and Non-Deterministic Finite Automata were developed as models for regular languages, with Kleene's theorem demonstrating their equivalence (Carroll & Long, 1989). Context-Free Grammars were introduced by Noam Chomsky and became foundational in programming language design (Chomsky, 1956). Pushdown Automata and Turing Machines have been instrumental in exploring decidable and undecidable problems, forming the basis of computational theory (Hopcroft & Ullman, 1979; Gandy & Yates, 2001). Similarly, the Tower of Hanoi problem has been widely used in teaching recursion and algorithm design.

4. Methodology

Approach

The approach for this project involves implementing a user-friendly Java-based application that demonstrates key concepts of automata theory and computational models. The program provides a graphical interface for simulating various automata models. Each model is designed to represent predefined languages or operations, allowing users to test and validate input strings, view state transitions, and observe the computational process in action.

Algorithms and Theoretical Models

The project relies on well-established algorithms and theoretical models from automata and formal language theory:

- **Deterministic Finite Automaton (DFA)**
  The DFA algorithm simulates state transitions based on the current state and input symbol. The algorithm tracks the string's acceptance by moving through states, and the transition table is displayed to show each step of the process.

- **Non-Deterministic Finite Automaton (NFA)**
  Similar to DFA, but with the ability to transition into multiple states for the same input symbol. This model allows for multiple computational paths, and the algorithm tracks all possible paths to determine if a string is accepted.

- **Context-Free Grammar (CFG)**
  The CFG model uses predefined production rules to validate whether a given string can be derived from the grammar. The algorithm applies these production rules in sequence to check for string acceptance.

- **Pushdown Automaton (PDA)**
  The PDA algorithm uses a stack to simulate context-free languages. For example, the algorithm checks if the input string conforms to the language a^nb^n (where $n \geq 1$) by pushing and popping symbols on the stack while processing the string.

- **Turing Machine**
  The Turing Machine implementation performs binary operations, such as addition and multiplication, using tape-based computation. The algorithm simulates the movement of the tape head and the transitions between states based on the input symbols.

- **Tower of Hanoi**
  The algorithm uses recursion to calculate the minimal number of moves required to solve the Tower of Hanoi problem. It generates and displays each move step-by-step.

Tools, Programming Languages, and Frameworks

The application is developed using Java and utilizes NetBeans IDE for development. The graphical user interface (GUI) is built using JFrame forms in NetBeans, leveraging the drag-and-drop interface design for ease of use. The program employs basic Java data structures such as arrays and stacks to model the state transitions and operations for each automaton. The GUI components, including buttons, text fields, combo boxes, and text areas, allow users to interact with the program and view real-time results.

5. Implementation

5.1. Deterministic Finite Automata (DFA)

Problem Definition

The Deterministic Finite Automata (DFA) implemented in this project is designed to simulate and validate binary strings for specific language criteria based on the structure and sequence of the input. The three DFA configurations are:

1. Starts with "10": Accepts strings that begin with the substring "10".
2. Contains "01": Accepts strings that contain the substring "01".
3. Ends with "11": Accepts strings that terminate with the substring "11".

These criteria represent classic problems in formal language theory, where the DFA ensures deterministic behavior for input processing.

Formal Description

A Deterministic Finite Automaton (DFA) is formally defined as a 5-tuple $M = (Q, \Sigma, \delta, q0, F)$, where:

- Q: A finite set of states.
- Σ: The input alphabet ($\{0, 1\}$ for binary strings).
- δ: The state transition function, defined as $\delta: Q \times \Sigma \rightarrow Q$.
- q0: The initial state, where the DFA begins.
- F: A set of accepting states. A string is accepted if the DFA ends in one of these states.

Specific DFA Configurations

A. DFA for Strings that Start with "10"

- Q = {q0, q1, q2, qd}: q0 is the start state, q2 is the accepting state, and qd is the dead state.
- Σ = {0, 1}: Input symbols are binary digits.

- Transition Function (δ):
  - δ (q0, 1) = q1, δ (q0, 0) = qd
  - δ (q1, 0) = q2, δ (q1, 1) = qd
  - δ (q2, 0) = q2, δ (q2, 1) = q2
  - δ (qd, 0) = qd, δ (qd, 1) = qd
- Start State: q0
- Accepting State: F = {q2}

B. DFA for Strings that Contain "01"
- Q = {q0, q1, q2, qd}: q0 is the start state, q2 is the accepting state, and qd is the dead state.
- Σ = {0,1}: Input symbols are binary digits.
- Transition Function (δ):
  - δ (q0, 0) = q0, δ (q0, 1) = q1
  - δ (q1, 0) = q2, δ (q1, 1) = q1
  - δ (q2, 0) = q2, δ (q2, 1) = q2
  - δ (qd, 0) = qd, δ (qd, 1) = qd
- Start State: q0
- Accepting State: F = {q2}

C. DFA for Strings that End with "11"
- Q = {q0, q1, q2, qd}: q0 is the start state, q2 is the accepting state, and qd is the dead state.
- Σ = {0,1}: Input symbols are binary digits.
- Transition Function (δ):
  - δ (q0, 0) = q0, δ (q0, 1) = q1
  - δ (q1, 0) = q0, δ (q1, 1) = q2
  - δ (q2, 0) = q0, δ (q2, 1) = q2
  - δ (qd, 0) = qd, δ (qd, 1) = qd
- Start State: q0
- Accepting State: F = {q2}

Implementation Process

The DFA is implemented using Java Swing for an interactive user interface. The process is divided into multiple steps:

1. Input Validation
   - The user enters a binary string.
   - The program verifies the input to ensure it contains only {0,1}.
   - The user selects one of the DFA criteria from a dropdown menu.
2. Transition Table Representation
   - A transition table is dynamically constructed based on the selected DFA criteria.
   - States and transitions are displayed in a table format.
3. Simulation
   - The input string is processed one symbol at a time using a Timer for step-by-step simulation.
   - At each step:
     - The current state and input symbol are evaluated.

        - The state transition function δ determines the next state.
    - The current state is logged, and transitions are visually updated.
4. Acceptance Check
    - After processing all input symbols, the DFA checks if the final state is in the set of accepting states (F).
    - A message displays whether the string is accepted or rejected.

Examples

A. Starts with "10":
    - Input: "1010"
    - Execution:
        1. Start at q0.
        2. Read '1': Transition to q1.
        3. Read '0': Transition to q2.
        4. Read '1': Remain in q2.
        5. Read '0': Remain in q2.
    - Result: Accepted.

B. Contains "01":
    - Input: "1001"
    - Execution:
        1. Start at q0.
        2. Read '1': Transition to q1.
        3. Read '0': Transition to q2.
        4. Read '0': Remain in q2.
        5. Read '1': Remain in q2.
    - Result: Accepted.

C. Ends with "11":
    - Input: "011"
    - Execution:
        1. Start at q0.
        2. Read '0': Remain in q0.
        3. Read '1': Transition to q1.
        4. Read '1': Transition to q2.
    - Result: Accepted.

5.2. Non-Deterministic Finite Automata (NDFA)

Problem Definition

The program models and simulates a Non-Deterministic Finite Automaton (NDFA) that accepts binary strings based on specific criteria. Two tasks are provided:

1. A string that contains the substring "101".
2. A string where the second-to-last digit is '1'.

Formal Description

A Non-Deterministic Finite Automata can be described as a 5-tuple: $M = (Q, \Sigma, \delta, q0, F)$, where:

- Q: A finite set of states.
- Σ: The input alphabet (binary in this case: {0,1}).

- δ: A transition function $\delta: Q \times \Sigma \rightarrow 2Q$ that maps a state and input to a set of possible next states.
- q0: The initial state.
- F: A set of accepting states.

How NDFA Differs from DFA

1. Non-determinism: In NDFA, for a given state and input symbol, there can be multiple next states (unlike DFA where each state-input pair maps to a single state).
2. State Transitions: NDFA allows parallel paths for processing inputs, while DFA processes inputs deterministically in one path.
3. Acceptance: NDFA accepts a string if at least one path reaches an accepting state.

Implementation Process

A. States and Transition Function:
   - States: q0, q1, q2
   - Transitions:
   - For "Contains '101'":
     q0 – 0 → q0
     q0 – 1 → q0, q1
     q1 – 0 → q2
     q2 – 1 → q2
   - For "Second-to-Last Digit is '1'":
     q0 – 0 → q0
     q0 – 1 → q0, q1
     q1 – 0 → q1, q2

B. Simulation Logic:
   - The automaton maintains a set of current states.
   - For each input symbol, it computes the next set of states based on transitions.
   - The string is accepted if any path reaches the final state q2.

Examples

A. Contains the substring "101"
   - Input: "101101"
   - Execution:
     1. Start at {q0}.
     2. Read '1': Transition to {q0, q1}.
     3. Read '0': Transition to {q0, q2}.
     4. Read '1': Transition to {q0, q1, q2}.
     5. Read '1': Transition to {q0, q1, q2}.
     6. Read '0': Transition to {q0, q2}.
     7. Read '1': Transition to {q0, q1, q2}.
   - Result: Accepted (Reached {q2}).

B. Second-to-last digit is '1'
   - Input: "11010"
   - Execution:
     1. Start at {q0}.
     2. Read '1': Transition to {q0, q1}.
     3. Read '1': Transition to {q0, q1}.
     4. Read '0': Transition to {q0, q1, q2}.
     5. Read '1': Transition to {q0, q1, q2}.
     6. Read '0': Transition to {q0, q1, q2}.
   - Result: Accepted (Reached {q2}).

5.3. Pushdown Automata (PDA)

Problem Definition

The Pushdown Automata (PDA) implemented in this project is designed to model and validate strings belonging to the language L = {a^n b^n | n ≥ 1}. This language consists of strings where the number of ‘a’ is equal to the number of ‘b’. The PDA uses a stack to keep track of the occurrences of 'a' and ensures they match the number of 'b' symbols in the input string. If the string satisfies the stack operations (matching 'a' with 'b') but does not meet the user-specified n constraint, it is rejected.

Formal Description

A Pushdown Automata (PDA) is formally defined as a 7-tuple: M = (Q, Σ, Γ, δ, q0, Z0, F) where:

- Q: A finite set of states.
- Σ: The input alphabet ({'a', 'b'} for this language).
- Γ: The stack alphabet.
- δ: The transition function.
- q0: The initial state.
- Z0: The initial stack symbol.
- F: A set of accepting states.

Stack Mechanism with *n* Validation

The stack operates as follows:

1. Start with an empty stack.
2. For each 'a' encountered, push 'a' onto the stack. Increment the counter for 'a'.
3. For each 'b' encountered, pop 'a' from the stack if the stack is not empty. Increment the counter for 'b'.
4. At the end of the input string, the PDA checks two conditions:
    - The stack is empty, indicating all ‘a’ have been matched with ‘b’.
    - The counts of a and b match n, as specified by the user.
5. If both conditions are met, the string is accepted. Otherwise, it is rejected.

Implementation Process

The PDA implementation has two main steps:

1. Input Validation:
    - The program prompts the user for two inputs:
        - n expected number of 'a' and 'b' in the language a^n b^n.
        - A test string to validate.
    - If the string's length is not 2n, it is immediately rejected.
2. Simulation:
    - Initialize counters for 'a' and 'b' to zero.
    - The stack is initialized as empty.
    - Process each character of the string:
        - For every 'a', push 'a' onto the stack and increment the 'a' counter.
        - For every 'b', pop 'a' from the stack and increment the 'b' counter.
    - After processing:
        - Check that the stack contains only the start marker.

- Verify that the counters for 'a' and 'b' both equal n.

If all conditions are satisfied, the string is accepted. Otherwise, it is rejected.

Examples

1. Input: n = 3, string: aaabbb
   Execution:
   1. In reading aaa, the stack transitions would be
      δ: (q0, aaabbb, [])
      δ: (q0, aabbb, [a])
      δ: (q0, abbb, [a, a])
      δ: (q0, bbb, [a, a, a])
      Stack after reading aaa: [a, a, a]
      Counters: countA = 3, countB = 0.
   2. In reading bbb, the stack transitions would be:
      δ: (q1, bbb, [a, a, a])
      δ: (q1, bb, [a, a])
      δ: (q1, b, [a])
      δ: (q1, ε, [])
      Stack after reading bbb: []
      Counters: countA = 3, countB = 3.
   3. Final Check:
      Stack is empty.
      Counters match n = 3.
   4. Result: Accepted.

5.4. Context-Free Grammar (CFG)

Problem Definition

Context-Free Grammars (CFGs) are a foundational tool in defining the syntax of programming and formal languages. However, validating strings against CFG production rules can be complex, especially for beginners. This project addresses the challenge by implementing a graphical user interface (GUI) that simplifies string validation based on specific CFG production rules. The application supports the following rules:

1. S -> aSb
2. S -> aSa | bSb | ε
3. S -> aSa | bSb | c

The system provides a user-friendly platform for selecting a rule, entering a string, and determining its validity, making CFG validation more accessible and intuitive.

Formal Description

A Context-Free Grammar (CFG) is formally defined as a 4-tuple (N, Σ, P, S), where:

- N (Non-terminals): Symbols that can be replaced (e.g., S).
- Σ (Terminals): Symbols that form the strings (e.g., a, b, c).
- P (Productions): Rules that describe how terminals and nonterminals combine.
- S (Start Symbol): The initial non-terminal symbol from which derivations begin (e.g., S)

Implementation Process

The CFG is implemented in Java Swing for an interactive interface that allows the user to input strings and visualize the parsing process. The process is divided into multiple steps:

A. Input Validation
   - Users enter a string for validation.
   - The program checks the input string to ensure it only contains valid terminal symbols (a, b, c) based on the selected production rule.

B. Product Rule Selection
   - Users select a production rule from a dropdown menu.
   - The application ensures that a rule is selected before proceeding with validation.

C. Validation Logic
   - Each CFG production rule has a corresponding recursive method to validate strings.
   - Ensures invalid inputs (e.g., empty strings or unsupported characters) are rejected.

D. Simulation
   - From the dropdown menu labeled "Select a Production Rule", choose one of the following:
     - SS→aSb
     - S→aSa|bSb|ε
     - S→aSa|bSb|c
   - In the text field labeled "Enter String to Validate", type the string to be tested. For example:
     - Input: abba
   - Click the "Validate String" button.
   - The application processes the input and checks it against the selected CFG production rule.
   - The result is displayed below the button:
     - Accepted: The string conforms to the production rule.
       - Example: For input abba and rule S→aSa|bSb|ε, the result will show "String is Accepted!" in green.
     - Rejected: The string does not conform to the rule.
       - Example: For input aba and ruleS→aSb, the result will show "String is Rejected!" in red.

Examples

A. Rule S -> aSb:
   a. Input: aabb
   b. Execution:
      1. Start with aabb.
      2. Match a and b, reducing to ab.
      3. Match a and b, reducing to an empty string.
      4. Result: Accepted.

B. Rule S -> aSa | bSb | ε:
   a) Input: abba
   b) Execution:
      1. Start with abba.
      2. Match a and a, reducing to bb.
      3. Match b and b, reducing to an empty string.
      4. Result: Accepted.

C. Rule S -> aSa | bSb | c:
   a. Input: c

b. Execution:
    1. Start with c.
    2. Recognize c as a valid base case.
    3. Result: Accepted.

5.5. Tower of Hanoi

Relation to Automata and Computational Theory

The Tower of Hanoi problem demonstrates fundamental concepts in computational theory, particularly recursion and state transitions, which are core to automata theory. It maps well to pushdown automata (PDA) because it involves a stack-based mechanism to track the recursive steps of moving disks between poles.

- States and Transitions:
    - Each step of the Tower of Hanoi problem can be seen as a transition between states, with disks, poles, and moves defining the configuration at any point. These transitions form a state space, much like finite automata.
- Recursive Nature:
    - The recursive approach models the problem by breaking it into smaller subproblems.
    - Pushdown automata use stacks to handle recursion, and the Tower of Hanoi similarly uses stacks to hold disks during intermediate steps.
- Complexity and Decision-Making:
    - The solution process explores a decision tree to find the optimal solution (minimum moves).

Solution Approach and Implementation

The Tower of Hanoi implemented in this project uses a recursive function to compute the steps required to solve the Tower of Hanoi problem. The GUI animates the solution by simulating the movement of disks between the poles and presenting the total number of moves. Furthermore, there are three key methods:

1. Recursive Algorithm:
    - Function towerOfHanoi(int n, String source, String target, String auxiliary):
        - If there is one disk, move it directly from the source to the target.
        - Otherwise:
            - Move n-1 disks from the source to the auxiliary pole.
            - Move the nth disk from the source to the target.
            - Move the n-1 disks from the auxiliary pole to the target.
2. Stacks to Represent Poles:
    - poleA, poleB, and poleC are stacks that hold the disks for the three poles.
3. GUI Updates:
    - Disk movements and pole states are animated in the TowerPanel class.
    - Moves are stored in the moveSteps array and executed one by one using a timer.

Examples

A. Input: n = 3
Initial State:
    - Pole A: [3, 2, 1]
    - Pole B: []

- Pole C: []

Move 1:

- Pole A: [3, 2]
- Pole B: []
- Pole C: [1]

Move 2:

- Pole A: [3]
- Pole B: [2]
- Pole C: [1]

Move 3:

- Pole A: [3]
- Pole B: [2, 1]
- Pole C: []

Move 4:

- Pole A: []
- Pole B: [2, 1]
- Pole C: [3]

Move 5:

- Pole A: [1]
- Pole B: [2]
- Pole C: [3]

Move 6:

- Pole A: [1]
- Pole B: []
- Pole C: [3, 2]

Move 7 (Final State):

- Pole A: []
- Pole B: []
- Pole C: [3, 2, 1]

Output: Steps: 7 moves, final state on pole C.

5.6. Turing Machine

Problem Definition

The Turing Machine implemented in this project was modeled to perform binary addition and unary multiplication. The GUI accepts two inputs and an operation, either addition (+) or multiplication (*), and simulates the corresponding Turing Machine computation.

Formal Description

A Turing Machine is formally defined as a 7-tuple (Q, T, B, ∑, δ, q0, F), where:

- Q is a finite set of states
- T is the tape alphabet
- B is the blank symbol (every cell is filled with B except input alphabet initially)
- ∑ is the input alphabet
- δ is a transition function which maps $Q \times T \rightarrow Q \times T \times \{L, R\}$. Depending on its present state and present tape alphabet (pointed by head pointer), it will move to a new state, change the tape symbol (may or may not), and move the head pointer to either left or right.
- q0 is the initial state

- F is the set of final states. If any state of F is reached, the input string is accepted.

a) For Binary Addition
   - Q = {q0, q1, q2, q3, q4, q5}
   - T = {0, 1, +, B}
   - ∑ = {0, 1, +}
   - Initial State: {q0}
   - Final State: {q5}

b) For Unary Multiplication
   - Q = {q0, q1, q2, q3, q4, q5, q6, q7, q8, q9, q10, q11, q12}
   - T = {1, 0, X, Y, B}
   - ∑ = {1, 0}

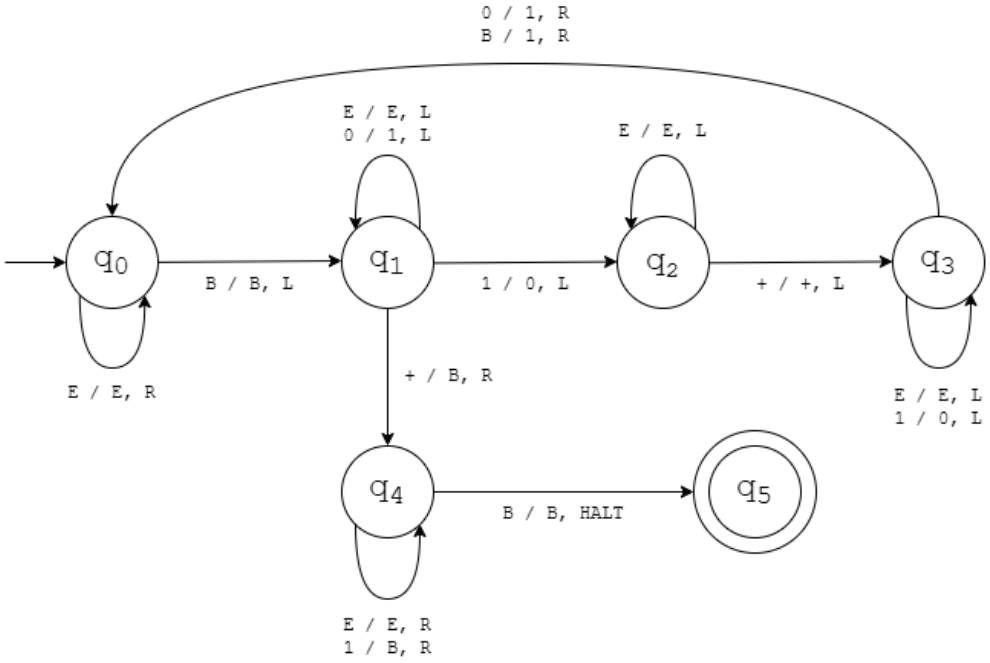
Implementation Process

The Turing Machine implementation has five main steps:

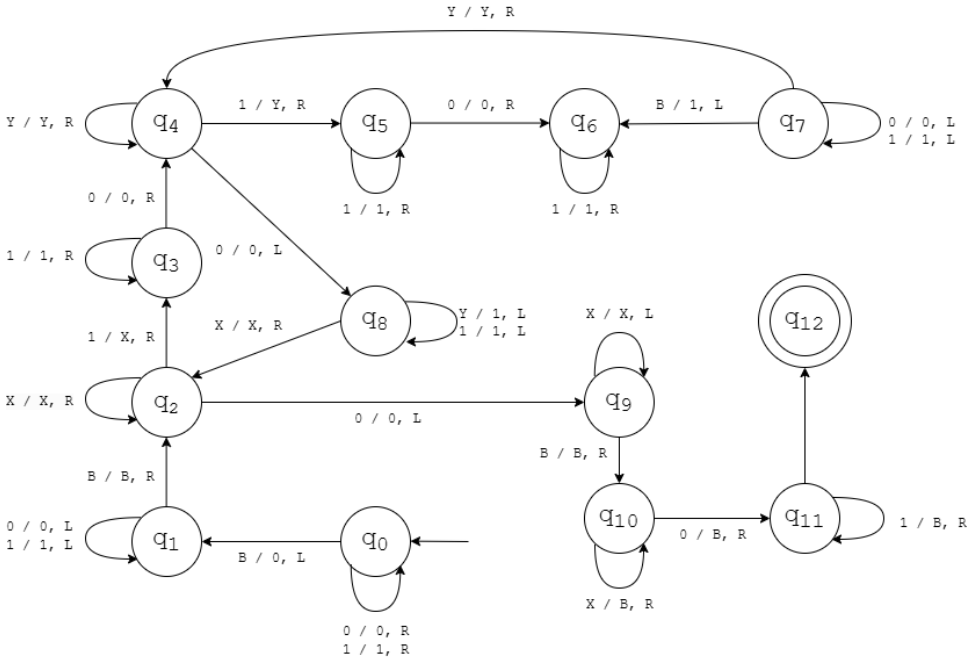
1. Input Preparation:
   - Binary addition inputs require numbers in binary representation. If the inputs are unequal in length, the shorter input will be padded by zeros to its left.
   - Unary multiplication inputs require numbers in unary representation.
2. Tape Initialization:
   - For addition:
     - The input is formatted as B<num1>+<num2>B.
   - For multiplication:
     - The input is formatted as B<num1>0<num2>B. Unary numbers are separated by a 0 indicating the multiplication operator.
3. Execution of Transition Function:
   - The machine simulates state transitions by reading the current tape symbol, updating it, moving the head, and switching states.
   - The while loop drives the computation, halting when the machine reaches a final state.
4. Output Generation:
   - For addition: The final binary result is written on the tape.
   - For multiplication: The tape contains the unary representation of the product.
5. GUI Representation:
   - Inputs and results are displayed in dedicated text areas (taInitialTape, taFinalTape, taResult).

State Diagrams

Binary Addition:



Unary Multiplication:



Examples

A. Input: num1 = 101 (binary for 5), num2 = 011 (binary for 3);
   Operation: Addition (+)
   Tape (Initial): B101+011B
   Transitions:
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 0) → δ (q0, 0, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, +) → δ (q0, +, R)
   δ (q0, 0) → δ (q0, 0, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, B) → δ (q1, B, L)
   ...
   Output: Binary Sum = 1000 (binary for 8)
B. Input: num1 = 11 (unary for 2), num2 = 1111 (unary for 4)
   Operation: Multiplication (*)
   Tape (Initial): B1101111B
   Transitions:
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 0) → δ (q0, 0, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, 1) → δ (q0, 1, R)
   δ (q0, B) → δ (q1, 0, L)
   ...
   Output: Unary Product = 11111111 (unary for 8)

6. Results and Analysis

The program successfully implements and demonstrates several computational models, including DFA, NFA, PDA, CFG, Turing Machine, and the Tower of Hanoi. Each model is encapsulated in an interactive simulation that allows users to input data, observe transitions, and see real-time processing of strings, production rules, and operations. It also provides clear visualizations that break down complex concepts into manageable steps, making abstract computational theories more tangible and understandable. Through this, users can explore the core principles of automata theory, formal grammars, and computational problems like binary operations and recursive algorithms.

The program fulfills its objectives by offering an accessible and interactive platform for users to experiment with key computational models. It effectively supports the testing and validation of strings, production rules, and operations, enabling users to understand how each model functions in practice. By using intuitive interfaces, the program bridges the gap between theoretical concepts and real-world applications, helping users grasp the behavior of automata and formal machines. Furthermore, the visual representation of string validation and problem-solving steps aids in conceptualizing the workings of complex computational models, enhancing both learning and experimentation.

An unexpected outcome we discovered was that for larger inputs, such as high n values in Turing Machine operations and Tower of Hanoi visualization, the step-by-step simulation slowed down due to GUI constraints or the visualization would not show up at all. To remedy

this, we have defined a maximum value for n that the program can handle efficiently without compromising performance. Optimizing for performance or offering a "fast-forward" option could also improve usability.

7. Conclusion

In conclusion, the program successfully models various automata and computational models, including DFA, NFA, CFG, PDA, Turing Machine, and the Tower of Hanoi. This achievement highlights the versatility and applicability of theoretical computation in solving and visualizing problems. By integrating multiple models, the program serves as an educational and research tool, bridging abstract theoretical concepts with practical computation.

The objectives of the project were largely achieved, as the program provides a comprehensive and interactive means to study automata and computational models. However, some limitations were encountered:

- Scalability Issues: For complex languages or large input sizes, the models might face efficiency constraints.
- Limited Functionality in CFG and PDA: While fundamental concepts are implemented, extending these to more complex grammar rules or pushdown operations remains challenging.
- UI/UX Limitations: The usability of the tool may need refinement for broader adoption by educators or researchers.

8. Future Work

Building on the success of this project, several improvements, extensions, and research directions can enhance its functionality and broaden its applications.

A. Improvements:
    - Enhanced Complexity Handling: Optimize algorithms for better performance with large or complex inputs.
    - Comprehensive Language Support: Extend CFG and PDA capabilities to support additional grammars and non-deterministic behaviors.

B. Extensions:
    - Additional Operations for Turing Machines: Implement subtraction and division as well as allow for inputs whether unary, binary, or decimal.
    - Automated Verification: Add features to verify the equivalence of automata (e.g., DFA minimization or NFA-to-DFA conversions).

C. Potential Applications:
    - Educational Tools: Develop this program into a comprehensive teaching resource for computer science curricula.
    - Research in Formal Methods: Use the program as a basis for experimenting with automata in formal verification, compiler design, and natural language processing.
    - Interdisciplinary Applications: Extend the Tower of Hanoi model to study recursive problem-solving or apply automata to biological computations (e.g., DNA sequencing).

D. Research Directions:
    - Quantum Automata Models: Investigate how quantum computing paradigms may extend the classical automata models.

- AI and Automata Integration: Explore the intersection of automata theory and machine learning, such as training neural networks to simulate or predict automaton behavior.

By addressing these directions, the project can evolve into a more robust platform for both theoretical exploration and practical applications.

9. References

Booth, T. L. (1967). *Sequential machines and automata theory*. John Wiley & Sons. https://doi.org/10.1093/comjnl/11.2.176

Carroll, J., and Long, D. 1989. *Theory of Finite Automata: with an Introduction to Formal Languages.* Prentice-Hall, Englewood Cliffs, NJ.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, *2*(3), 113–124. https://doi.org/10.1109/TIT.1956.1056813

Gandy, R. O., & Yates, C. E. M. (Eds.). (2001). *Collected works of A. M. Turing: Mathematical logic*. Elsevier Science.

Hopcroft, J. E., & Ullman, J. D. (1979). *Introduction to automata theory, languages, and computation.* Addison-Wesley.

OpenAI. (2024). *ChatGPT*. Retrieved from https://openai.com

Sipser, M. (1996). *Introduction to the theory of computation*. PWS Publishing.

Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, *42*(2), 230–265. https://doi.org/10.1112/plms/s2-42.1.230